

CLASS 17+18 MINI PROJECT React Router + Data Fetching

XELI AI Labs • Weeks 5–6 • Due: End of Week 6

8–10 hours	Choose 1 of 3 options	■ AI tools encouraged
------------	-----------------------	-----------------------

■ What This Is

This is your biggest project yet. It uses everything: components, state, useEffect, fetch, .map(), conditional rendering, AND React Router. Pick the option that excites you most — all three are the right difficulty.

Every Option Must Have These — Non-Negotiable

■ The Standard

These requirements are the same for all three project options. Think of them as the checklist your senior developer checks before the code review.

Requirement	What It Means
Tech Stack	Vite + React + React Router. All three. No exceptions.
Home / List Page	Fetches data from a real API and displays items as cards.
Card Content	Each card shows at least 3 fields (image, title, and one other).
Detail Page	A separate page at a dynamic route like /movie/:id or /recipe/:id.
Navbar	Works on every page. Active page is highlighted using NavLink.
Loading State	Show "Loading..." or a spinner on every single API call.
Error State	Show a helpful message if the API fails. With a retry button.
404 Page	A catch-all route for invalid URLs. Links back to home.
Responsive	Works on mobile and desktop. Test at 375px width.
GitHub	Pushed to GitHub with a README explaining the project.

■ Option 1: Movie Database App

■ In Plain English

Browse popular movies, search by title, and click into individual movie pages. The most interview-impressive option — movie apps are a classic frontend portfolio piece.

REQUIREMENTS

- Browse popular movies on the home page (fetched from API)
- Search movies by title with a search bar
- Movie cards: poster image, title, star rating
- Click any card → detail page at /movie/:id
- Detail page: full plot summary, release date, genre list, cast overview
- Navbar: Home, Search — with active highlighting
- Loading + error states on every API call

API SETUP — TMDb (free, requires key)

```
// 1. Sign up at themoviedb.org // 2. Settings → API → Create → get your key // 3. Store it in .env:  
VITE_TMDB_KEY=your_key_here // Popular movies:  
https://api.themoviedb.org/3/movie/popular?api_key=${import.meta.env.VITE_TMDB_KEY} // Search:  
https://api.themoviedb.org/3/search/movie?api_key=...&query=Wakanda // Movie detail (use movie.id from the  
list): https://api.themoviedb.org/3/movie/550?api_key=... // Poster image base URL:  
https://image.tmdb.org/t/p/w500 + poster_path
```

BONUS FEATURES

- ★ Watchlist: save/remove movies to localStorage. Show saved list on a /watchlist page
- ★ Filter by genre using the /genre/movie/list endpoint
- ★ Show similar movies at the bottom of each detail page

■ "I'm building a Movie Database app with React, Vite, and React Router using the TMDb API. Help me plan the full component tree and write the App.jsx route setup."

■ ■ Option 2: Recipe Finder App

■ In Plain English

Search recipes, browse by category, and view full cooking instructions. Uses TheMealDB — completely free with no API key required. Great choice for beginners.

REQUIREMENTS

- Home page shows featured/random meals as cards
- Search recipes by name or main ingredient
- Recipe cards: meal image, name, category
- Click any card → detail page at /recipe/:id
- Detail page: ingredients list, step-by-step instructions, category + area tags
- Filter by meal category (Chicken, Seafood, Vegetarian, etc.)
- Navbar: Home, Search, Categories

API SETUP — TheMealDB (100% free, no key)

```
// Search by name: https://www.themaldb.com/api/json/v1/1/search.php?s=chicken // Single random meal (for featured section): https://www.themaldb.com/api/json/v1/1/random.php // Meal by ID (for detail page): https://www.themaldb.com/api/json/v1/1/lookup.php?i=52772 // Filter by category: https://www.themaldb.com/api/json/v1/1/filter.php?c=Seafood // List all categories: https://www.themaldb.com/api/json/v1/1/categories.php
```

BONUS FEATURES

- ★ Favourites: save/remove recipes to localStorage
- ★ "Surprise me!" button that fetches and redirects to a random recipe
- ★ Ingredient list with checkboxes for cooking mode

■ "I'm building a Recipe Finder using the TheMealDB API. No API key required. Help me plan the component tree and write the initial App.jsx routes."

■ Option 3: Country Explorer (No API Key)

■ In Plain English

Browse African (or world) countries, filter by region, and view detailed info for each one. Zero setup friction — open the API URL and you're already fetching data. Great for getting straight into building.

REQUIREMENTS

- Home page: grid of country cards (start with Africa or world)
- Country cards: flag, name, capital, population
- Search/filter countries by name as you type
- Filter by sub-region (West Africa, East Africa, North Africa, etc.)
- Click any card → detail page at /country/:name
- Detail page: languages, currency, area, timezone, flag (large)
- Show bordering countries on the detail page (as clickable links!)

API SETUP — REST Countries (completely free, no key)

```
// All African countries: https://restcountries.com/v3.1/region/africa // All countries worldwide:
https://restcountries.com/v3.1/all // Specific country by name: https://restcountries.com/v3.1/name/Nigeria
// Only fetch the fields you need (faster): https://restcountries.com/v3.1/region/africa
?fields=name,flags,capital,population,subregion // Country by alpha code (for border links):
https://restcountries.com/v3.1/alpha/NGA
```

BONUS FEATURES

- ★ Sort countries by population (ascending/descending) or alphabetically
- ★ Dark/light mode toggle that persists to localStorage
- ★ Show a map link (Google Maps or OpenStreetMap) for each country

■ "I'm building a Country Explorer using the REST Countries API. No key needed. Help me design the component tree and write the skeleton App.jsx with React Router."

Grading Rubric

Area	What We Look For	Pts
React Router	BrowserRouter in main · Routes + Route · NavLink active state · dynamic :id route · 404 page	30
Data Fetching	useEffect with [] · loading state · error state · refresh/retry works	25
Component Design	pages/ vs components/ folders · components are reusable · state lives in right place	20
UI & UX	Responsive · cards show 3+ fields · detail page has full content · empty states handled	15
Code Quality	No direct state mutation · key props present · no unused imports · readable code	10

Suggested Timeline

Night of Class 17+18	Finish route skeleton + Navbar + home page fetching data
Day 2	Complete home page UI (cards styled). Start detail page.
Day 3	Detail page working with useParams. Error + loading states.
Day 4	Search / filtering. Responsive design.
Day 5	Polish, README, push to GitHub, test on mobile.
Day 6 (buffer)	Bonus features if ahead. Rest if on track.

■ Stuck on any day? Paste your current component + the specific problem into Claude. Use the 4-part prompt: context + goal + problem + code. Understand every answer before using it.

This project is your proof of concept. Build something you are proud to show. ■ ■ ■ ■